

ROCT · SMARTFRIDGE

User Stories

Documentation finale — fonctionnalités réellement implémentées

Format : « En tant que [utilisateur], je veux [action], afin de [objectif]. »

Aucune histoire inventée : chaque story correspond à un parcours réellement réalisable dans le code actuel.

Personnes concernées

 Utilisateur

adhérent du réfrigérateur (compte « user »)

 Administrateur

compte « admin » (gestion globale)

1. Authentification

S'inscrire

En tant que **visiteur**, je veux **créer un compte** avec un nom, un email et un mot de passe (avec confirmation), afin d'obtenir mon espace personnel dans le réfrigérateur.

Réalité : page `Register` + validations + `POST /api/auth/register` ; un message de bienvenue est créé côté serveur (type `welcome`). L'inscription ne connecte pas automatiquement : l'utilisateur se connecte ensuite via `/login`.

Se connecter

En tant que **utilisateur**, je veux **me connecter** avec mon email et mon mot de passe, afin d'accéder à mon tableau de bord et à mes produits.

Réalité : `POST /api/auth/login` → JWT (1 jour) stocké dans `localStorage`, redirection vers `/dashboard`. Un compte désactivé est refusé (403).

Se déconnecter

En tant que **utilisateur**, je veux **me déconnecter**, afin de protéger l'accès à mon espace depuis un appareil partagé.

Réalité : bouton « Déconnexion » de la Sidebar → `POST /api/auth/logout` + suppression du token local + retour à la page de connexion.

Basculer entre connexion et inscription

En tant que **visiteur**, je veux **passer de la connexion à la création de compte** (et inversement), afin de commencer l'utilisation sans perdre de temps.

Réalité : liens `/register` sur Login et `/login` sur Register.

2. Profil utilisateur

En tant que **utilisateur**, je veux **consulter mon profil** (nom, email, rôle), afin de vérifier les informations de mon compte.

En tant que **utilisateur**, je veux **modifier mon nom et mon email** (le mot de passe actuel étant requis pour changer l'email), afin de garder mes coordonnées à jour.

Réalité : page `Profile` → `PUT /api/auth/profile` .

3. Gestion des produits

En tant que **utilisateur**, je veux **ajouter un produit manuellement** (nom, catégorie, dates d'achat/expiration, seuil d'alerte, quantité, unité, prix), afin de suivre le contenu de mon réfrigérateur.

Réalité : page `Products` (mode « manuel ») → `POST /api/products` .

En tant que **utilisateur**, je veux **scanner un code-barres** avec la caméra, afin de renseigner automatiquement un produit sans tout saisir.

Réalité : librairie `html5-qrcode` + `GET /api/products/barcode/:code` (Open Food Facts) → préremplissage du formulaire. À vérifier — pertinence des données (catégorie mappée) selon la disponibilité dans la base.

En tant que **utilisateur**, je veux **consulter mes produits** avec leur statut, afin d'avoir une vue d'ensemble de mon réfrigérateur.

Réalité : liste de la page `Products` et du `Dashboard` (`GET /api/products`).

En tant que **utilisateur**, je veux **rechercher un produit par son nom**, afin de le retrouver rapidement dans une longue liste.

En tant que **utilisateur**, je veux **filtrer les produits par catégorie**, afin de ne voir que les produits qui m'intéressent.

En tant que **utilisateur**, je veux **consulter les détails d'un produit** (quantité initiale/restante, unité, prix, dates, seuil d'alerte), afin de suivre précisément son état.

Réalité : page `ProductDetails` → `GET /api/products/:id` .

En tant que **utilisateur**, je veux **modifier un produit**, afin de corriger une information (quantité, dates, prix, etc.).

Réalité : formulaire inline sur `ProductDetails` → `PUT /api/products/:id` . À vérifier — la page `EditProduct` (/products/:id/edit) reste routée mais n'est plus atteignable depuis l'interface (le bouton « Modifier » ouvre le formulaire inline).

En tant que **utilisateur**, je veux **supprimer un produit** (avec confirmation), afin de retirer de ma liste un produit que je n'ai plus.

Réalité : `DELETE /api/products/:id` puis retour à la liste.

En tant que **utilisateur**, je veux **consommer une quantité d'un produit**, afin de suivre ce que j'utilise et la valeur consommée.

Réalité : `PATCH /api/products/:id/consume` → décrémentation de la quantité + enregistrement `Consumption` + affichage du montant consommé (DH).

4. Expiration & statuts

En tant que **utilisateur**, je veux **connaître automatiquement le statut de chaque produit** (valide / bientôt expiré / expiré), afin de savoir quoi consommer en priorité.

Réalité : statut calculé à partir des jours restants (< 0 expiré, $\leq$ `expirationAlertDays` bientôt expiré, sinon valide).

En tant que **utilisateur**, je veux **recevoir une alerte avant l'expiration** d'un produit, afin de le consommer à temps.

Réalité : `node-cron` (00:00 + au démarrage) → notifications `bientot_expire` ; rappel toutes les 24 h tant que non lue.

En tant que **utilisateur**, je veux **être prévenu quand un produit est expiré**, afin de retirer les aliments périmés.

Réalité : notification unique `expire` par produit.

5. Notifications

En tant que **nouvel utilisateur**, je veux **recevoir une notification de bienvenue**, afin d'apprendre à utiliser l'application.

En tant que **utilisateur**, je veux **consulter la liste de mes notifications** (avec date d'envoi relative), afin de rester informé des expirations.

Réalité : page `Notifications` → `GET /api/notifications` .

En tant que **utilisateur**, je veux **filtrer les notifications par date**, afin de retrouver une alerte d'un jour précis.

Réalité : paramètre `?date=AAAA-MM-JJ` .

En tant que **utilisateur**, je veux **marquer une notification comme lue** et connaître le nombre de non-lues, afin de faire le tri rapidement.

Réalité : `PATCH /api/notifications/:id/read` + compteur dans l'interface.

6. Tableau de bord

En tant que **utilisateur**, je veux **voir les statistiques de mon réfrigérateur** (total, valides, bientôt expirés, expirés), afin de saisir sa santé d'un coup d'œil.

En tant que **utilisateur**, je veux **voir les produits « expirant bientôt » et « expirés »** (jours restants / jours écoulés), afin d'agir rapidement.

En tant que **utilisateur**, je veux **consulter mes insights mensuels** (valeur consommée et pertes alimentaires), afin de mesurer mon gaspillage.

Réalité : `GET /api/consumptions/monthly` + `GET /api/losses/monthly` . À vérifier — la carte « Consommation » affiche la valeur (DH) sans suffixe « DH ».

En tant que **utilisateur**, je veux **retrouver tous mes produits sur le tableau de bord** avec recherche et filtre, afin d'avoir une vue unique.

7. Catégories

En tant que **utilisateur**, je veux **choisir une catégorie parmi celles existantes** à la création / modification d'un produit, afin de classer mes aliments.

Réalité : `GET /api/categories` alimente les listes déroulantes.

En tant que **administrateur**, je veux **créer, renommer ou supprimer des catégories**, afin de maintenir un référentiel pertinent.

Réalité : `POST / PUT / DELETE /api/categories` (admin) depuis la page `Profile` .

8. Administration (espace admin)

En tant que **administrateur**, je veux **voir les statistiques globales** (nombre d'utilisateurs, de produits et de produits expirés), afin de superviser l'application.

Réalité : `GET /api/admin/statistics` .

En tant que **administrateur**, je veux **gérer les comptes utilisateurs** (activer / désactiver), afin de contrôler qui peut se connecter.

Réalité : `GET /api/admin/users` + `PUT /api/admin/users/:id/status` . Un compte désactivé ne peut plus se connecter.

Limites & points d'attention (fidélité au code)

Sujet	Constats réels
Récupération de mot de passe « oublié »	Non implémentée (aucune route, aucun email).
Garde de routes frontend	Non implémentée : aucune protection des routes d'application par composant guard.
Page EditProduct	Router présente mais plus accessible via l'interface (édition inline privilégiée).
Inscription automatique	Après l'inscription, l'utilisateur est invité à se connecter (pas de connexion automatique).
Mapping Open Food Facts → catégories	Identifiants de catégories codés en dur dans le service ; dépend de catégories pré-existantes en base.
Dashboard « Consumption »	Valeur affichée = somme des montants consommés (en DH) ; libellé « produits consommés », sans suffixe monétaire.
Validation profil (updateProfileSchema)	Schéma Zod défini mais non appliqué sur PUT /api/auth/profile.

Note : toutes les stories « Partiellement implémentées » ou « À vérifier » sont signalées explicitement ci-dessus. La liste ci-dessus ne contient que des parcours réellement réalisables dans la version actuelle du code.

ROCT · SmartFridge — User Stories rédigées à partir du code source actuel du dépôt github.com/tourabikhadija/SmartFridge.